# FINAL PROJECT

## ANALYSIS OF ACTIVITY IN SECURITY LOGS


**Submitted to**

Prof. Dr. James Tsengching Shen

Department of Computer Science

College of Engineering and Computer Science (ECS)

California State University, Fullerton (CSUF)

for the fulfillment of the requirements for

***CPSC 531: Advanced Database Management***

**TEAM MEMBERS**

**MANAV KUMAR M (**CWID: **869213850)**

**ZARIF SHAMS (**CWID: **886995554)**

## INTRODUCTION

Cybersecurity has become an essential pillar in the digital age, where organizations generate and rely on massive volumes of system and network activity logs. These logs serve as crucial evidence for monitoring system integrity, detecting security incidents, and understanding user behavior. However, the unstructured and high-velocity nature of log data presents significant challenges for conventional analysis methods.

This project harnesses the capabilities of big data technologies to extract insights from security logs. By leveraging scalable storage (HDFS) and distributed computing (Apache Spark), the study aims to efficiently process, parse, and analyze large volumes of log data to identify anomalies, track IP-level threats, and visualize attack distributions.

Traditional log management tools often fall short in delivering real-time threat detection or scalable analytics. Through this research, we employ advanced preprocessing, structured transformations, and interactive visualizations to build a comprehensive framework for cyberlog analysis — enabling better situational awareness, proactive defense mechanisms, and data-driven security decisions.

## PROJECT GOALS

The core aim of this project was to design a local data pipeline that utilizes Hadoop Distributed File System (HDFS), YARN, and Apache Spark for analyzing a cybersecurity log dataset. The analytical results are displayed through a Streamlit-powered web application. This implementation showcases the effectiveness of distributed computing tools for processing and visualizing large datasets within a single-node architecture

## PROJECT ABSTRACT

In today's interconnected digital landscape, organizations generate enormous volumes of system and security logs that hold vital information regarding user activity, application behavior, and potential security threats. However, the sheer size, velocity, and unstructured nature of these logs make it increasingly difficult to extract actionable insights using traditional methods.

This project harnesses the power of big data analytics to explore and analyze cybersecurity logs for detecting patterns, anomalies, and potential breaches. By integrating scalable technologies such as Hadoop HDFS for storage and Apache Spark for distributed processing, the study enables efficient handling and transformation of high-volume log data.

Through advanced data preprocessing, visualization, and pattern recognition techniques using PySpark and Streamlit, the project aims to uncover hidden trends in user behavior, identify frequent attack sources, and evaluate system performance under different threat scenarios. This comprehensive approach provides a deeper understanding of cybersecurity events, paving the way for real-time monitoring, threat prediction, and enhanced security decision-making.

## DATASET OVERVIEW

| | A | B | C | D | E | F | G | H | I | J | K | L |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Timestamp | Source IP Addre | Destination IP A | Source Port | Destination Port | Protocol | Packet Length | Packet Type | Traffic Type | Payload Data | Malware Indicators | Anomaly Scores |
| 2 | 2023-05-30 6:33 | 103.216.15.12 | 84.9.164.252 | 31225 | 17616 | ICMP | 503 | Data | HTTP | Qui natus odio asperiores nam. Op<br>Maiores possimus ipsum saepe vita | IoC Detected | 28.67 |
| 3 | 2020-08-26 7:08 | 78.199.217.198 | 66.191.137.154 | 17245 | 48166 | ICMP | 1174 | Data | HTTP | Aperiam quos modi officiis veritatis<br>Illo animi mollitia vero voluptates er | IoC Detected | 51.5 |
| 4 | 2022-11-13 8:23 | 63.79.210.48 | 198.219.82.17 | 16811 | 53600 | UDP | 306 | Control | HTTP | Perferendis sapiente vitae soluta. H | IoC Detected | 87.42 |
| 5 | 2023-07-02 10:3 | 163.42.196.10 | 101.228.192.25 | 20018 | 32534 | UDP | 385 | Data | HTTP | Totam maxime beatae expedita explicabo porro labore. Minima ab fugit | | 15.79 |
| 6 | 2023-07-16 13:1 | 71.166.185.76 | 189.243.174.23 | 6131 | 26646 | TCP | 1462 | Data | DNS | Odit nesciunt dolorem nisi iste iusto. Animi voluptates soluta quis dolori<br>Quia illo fugit eligendi doloremque. In doloremque autem iure. | | 0.52 |
| 7 | 2022-10-28 13:1 | 198.102.5.160 | 147.190.155.13 | 17430 | 52805 | UDP | 1423 | Data | HTTP | Repellat quas illum harum fugit incidunt exercitationem illum. Voluptate | | 5.76 |
| 8 | 2022-05-16 17:5 | 97.253.103.59 | 77.16.101.53 | 26562 | 17416 | TCP | 379 | Data | DNS | Qui numquam inventore repellat ratione fugit odit. Quidem est possimu<br>Excepturi quisquam iusto provident adipisci. | | 31.55 |
| 9 | 2023-02-12 7:13 | 11.48.99.245 | 178.157.14.116 | 34489 | 20396 | ICMP | 1022 | Data | DNS | Amet libero optio quidem praesentiu<br>Eveniet dolor odio libero. Minus iste | IoC Detected | 54.05 |
| 10 | 2023-06-27 11:0 | 49.32.208.167 | 72.202.237.9 | 56296 | 20857 | TCP | 1281 | Control | FTP | Veritatis nihil amet atque molestias | IoC Detected | 56.34 |
| 11 | 2021-08-15 22:2 | 114.109.149.113 | 160.88.194.172 | 37918 | 50039 | UDP | 224 | Data | HTTP | Consequatur ipsum autem reprehenderit quae. Doloribus dicta laborios | | 16.51 |
| 12 | 2022-07-20 13:2 | 177.21.83.200 | 196.218.124.16 | 35538 | 35006 | ICMP | 661 | Data | HTTP | Sequi maxime voluptate ea. Eius officiis eaque. Nesciunt aspernatur re<br>Eius dolore molestias. Voluptas laboriosam vero suscipit assumenda in | | 24.91 |
| 13 | 2022-06-26 15:1 | 92.4.25.171 | 112.43.185.24 | 10903 | 36817 | TCP | 281 | Control | HTTP | Nihil praesentium asperiores omnis ullam libero. Facilis eius aspernatu | | 86.07 |
| 14 | 2020-09-30 21:3 | 57.91.207.84 | 98.96.110.38 | 53471 | 38048 | ICMP | 64 | Control | DNS | Earum sit est et eaque ipsam. Vero repellendus error iusto eveniet. | | 74.2 |
| 15 | 2021-01-13 2:30 | 80.28.21.123 | 111.204.103.106 | 43729 | 10845 | ICMP | 1341 | Data | HTTP | Ab voluptatibus nam aperiam. Est sed nostrum animi quae quas. Rerur<br>Cupiditate provident doloremque aliquam maiores tenetur tempora.<br>Similique dolor ullam illo placeat. | | 62.14 |
| 16 | 2023-02-01 13:1 | 54.163.130.178 | 62.112.149.214 | 47795 | 35243 | UDP | 913 | Data | DNS | Sit doloremque explicabo. Cupiditate placeat exercitationem. Dolores r<br>Suscipit itaque inventore eveniet ipsa. Ea ipsa minima impedit aliquam | | 72.65 |
| 17 | 2023-09-15 20:3 | 130.163.192.25 | 12.192.2.112 | 24912 | 21176 | TCP | 425 | Data | DNS | Voluptates nam doloremque. Eius amet officiis aperiam delectus molliti | | 91.56 |
| 18 | 2023-04-18 16:4 | 4.255.187.165 | 136.159.186.23 | 39934 | 5259 | TCP | 838 | Control | HTTP | Sed facere culpa aliquam. Adipisci error quisquam reprehenderit itaque | | 43.83 |

## PROJECT FUNCTIONALITIES

Here are concise one-line descriptions for each functionality in your cyberlog analysis project:

- Store cybersecurity logs (cyberlogs.csv) in HDFS:

Upload and store raw cyberlog data in the Hadoop Distributed File System for scalable access.

- Process the log data using PySpark and Apache Spark:

Utilize PySpark to perform distributed data processing on large log datasets.

- Use YARN as the resource manager for Spark jobs:

Manage and schedule Spark processing tasks using YARN for efficient resource allocation.

- Perform simple data aggregations and column selections in PySpark before presenting them in the frontend:

Clean and transform the data using PySpark by selecting relevant fields and computing aggregations.

- Present data through a Streamlit web application:

Display analyzed results through an interactive dashboard built using Streamlit.

- Allow end-users to view and explore the logs via a web interface:

Enable users to interactively explore log patterns and summaries via a user-friendly web UI.

PROJECT ARCHITECTURE

Architecture and Design : This project follows a modular, single-node architecture built entirely on a Linux machine. The key components are:

- HDFS (Hadoop Distributed File System):

- Used for storing the CSV data file reliably and in a scalable way.
- → *Ensures fault-tolerant and distributed storage for high-volume log data.*
- YARN (Yet Another Resource Negotiator):

Handles resource management and job scheduling for Spark tasks.

→ *Optimizes execution by managing compute resources across Spark processes.*

- Apache Spark:

Performs data loading, transformation, and analysis on the CSV dataset.

→ *Enables fast, in-memory distributed data processing for large-scale log files.*

- PySpark and Streamlit App:

The project integrated PySpark data analysis and interactive Streamlit visualizations within a single Python file. This unified application read and analyzed cybersecurity logs from HDFS using Spark and then rendered dynamic charts and tables via Streamlit.

→ *Delivers an end-to-end pipeline from log ingestion to real-time dashboard visualization in a single interface.*

DATA FLOW

Data is uploaded to HDFS using Hadoop commands.
→ *This ensures scalable and persistent storage of the cyberlogs for processing.*

Spark reads the data from HDFS mainly using a PySpark script.
→ *The distributed computing engine fetches the logs efficiently for transformation.*

The PySpark Script performs a limited analysis (e.g., schema inspection, record filtering, Data Aggregation).
→ *Initial processing helps clean, filter, and summarize data for visualization.*

The results are passed to the Streamlit application and charts get displayed.
→ *Visual insights like tables, graphs, and metrics are presented on a dynamic web UI.*

## DOCUMENTATION

Deployment instructions and instructions to run the application are shown the the Github

ReadMe File,

Github: https://github.com/Zarifzz/hdfs-spark-project

## DISCUSSION OF THE RESULTS

This project effectively showcased the complete workflow—from ingesting data into HDFS, utilizing YARN for resource management, analyzing the data using PySpark, to delivering the results through an interactive Streamlit web application—all orchestrated within a single Python script.

This streamlined design demonstrated an efficient use of Big Data tools and enabled rapid prototyping.

Key outcomes include:

- The Streamlit dashboard successfully captured the structure and patterns within the cybersecurity log dataset, offering column-wise summaries, insightful visualizations, and clear security-related findings.
- The PySpark script efficiently accessed and processed the cyberlogs.csv file directly from HDFS, confirming the seamless integration of Spark with distributed storage.
- Analytical insights derived from the PySpark pipeline highlighted trends and anomalies in the log data, which were effectively communicated through real-time visuals in the Streamlit frontend.
- System performance remained consistently stable while handling approximately 40,000 records, with minimal latency observed in both backend processing and frontend rendering.